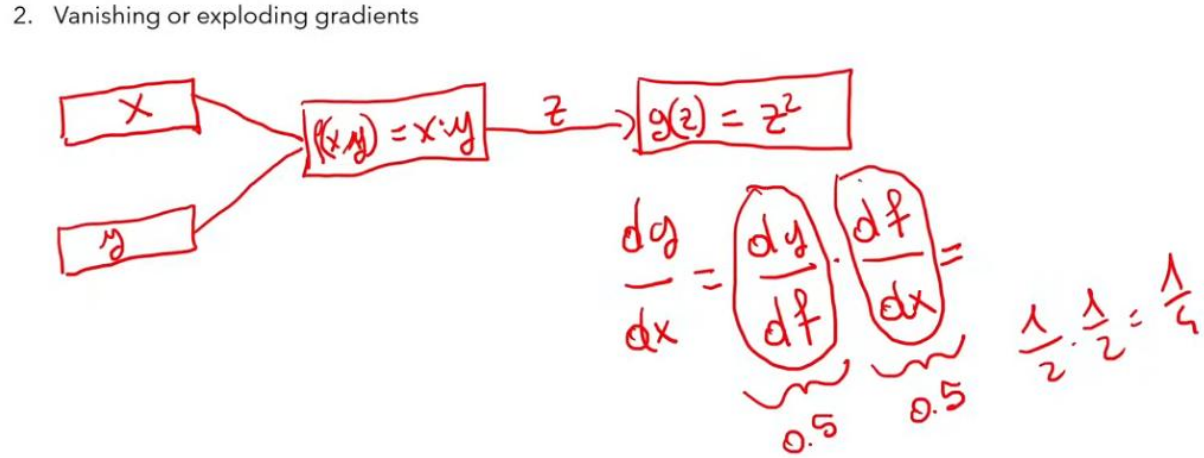
Attention is all you need (Transformer) - Model explanation (including math), Inference and Training

The second problem was the **vanishing or exploding gradients.**
Frameworks like PyTorch convert our networks into a computation graph. So basically, suppose we have a computation graph. This is not a neural network - I will be making a very simple computational graph that has nothing to do with neural networks, but it will help illustrate the problem.
Imagine we have two inputs, $X$and another input, let's call it Y. Our computational graph first multiplies these two numbers. So, we have a function $f(X,Y)$that is equal to Xmultiplied by Y. Let the result be called Z.
This result is then mapped to another function, let's call it $G(Z)$, which is equal to Z^2.
What PyTorch, for example, does is calculate derivatives. Usually, we have a loss function, and PyTorch calculates the derivative of the loss function with respect to each weight. In this simplified case, we just calculate the derivative of the output function with respect to all of its inputs.
So the derivative of $G$with respect to $X$is equal to the derivative of $G$with respect to f, multiplied by the derivative of $f$with respect to X. This is called the chain rule.
Now, as you can see, the longer the chain of computation—if we have many nodes one after another—the longer this multiplication chain becomes. In our example, the distance from the input to the output is two but imagine if it were 100 or 1000.
Now imagine each of these values is 0.5. When we multiply them together, the resulting number becomes smaller than the initial numbers. For example, 0.5 multiplied by 0.5 equals 0.25. If we keep multiplying numbers smaller than one, the result becomes smaller. On the other hand, if the numbers are greater than one, multiplying them produces a number larger than any of the individual values.
So if we have a very long chain of computations, the result will eventually either become a very small number or a very large number. This is not desirable.
First, our CPU or GPU can only represent numbers up to a certain precision, such as 32-bit or 64-bit. If the number becomes too small, its contribution to output becomes negligible. When PyTorch or any automatic differentiation framework calculates how to adjust the weights, the weights will change very, very slowly because the gradient contribution is extremely small. This means the gradient is vanishing.
In the opposite case, the values can explode and become very large numbers, resulting in exploding gradients.



The third problem is difficulty in **accessing information from long time ago**. What does it mean? We saw that the first input token is given to the recurrent neural network along with the first state.
Now we need to think that the recurrent neural network is a long graph of computation. It will produce a new hidden state, then we will use the new hidden state along with the next token to produce the next output.
If we have a very long sequence of input tokens, the last token will have a hidden state whose contribution from the first token has nearly gone because of this long chain of multiplication. So, the last token will not depend much on the first token.
This is also not good because, for example, we know as humans that in a text—especially quite long text—the context that we saw, let's say 200 words before, is still relevant to the context of the current words. This is something that the RNN could not model well.

Input Embeddings

We have a sentence of, in this case, six words. We tokenize it - we transform the sentence into tokens.
What does it mean to tokenize? We split the sentence into single words. It is not necessary to always split the sentence using single words. We can even split the sentence into parts that are smaller than a single word. So, we could even split this sentence into, let's say, 20 tokens by splitting each word into multiple parts. This is usually done in most modern Transformer models, but we will not be doing it here.
So let's suppose we have this input sentence, and we split it into tokens, and each token is a single word.
The next step we do is we map these words into numbers, and these numbers represent the position of these words in our vocabulary. Imagine we have a vocabulary of all the possible words that appear in our training set. Each word will occupy a position in this vocabulary. For example, the word *"your"* will occupy position 105, the word "*cat"* will occupy position 6587, etc.
As you can see, this "*cat"* here has the same number as this "*cat"* here because they occupy the same position in the vocabulary.
We take these numbers, which are called - input IDs, and we map them into a vector of size 512. This vector is made of 512 numbers, and we always map the same word to the same embedding. However, this embedding is not fixed—it is a parameter of our model.
So our model will learn to change these numbers in such a way that they represent the meaning of the word. The input ID never changes because our vocabulary is fixed, but the embedding will change throughout the training process of the model. The embedding numbers will change according to the needs of the loss function.
So the input embeddings are basically mapping our single word into an embedding of size 512. We call this quantity $d_{model} = 512$, which is the same name used in the paper *Attention Is All You Need*.

Positional Encoding

What we want is that each word should carry some information about its position in the sentence. Now we have built a matrix of words that are embeddings, but they don't convey any information about where that particular word is inside the sentence.

This is the job of the positional encoding. What we want is for the model to treat words that appear close to each other as close, and words that are distant as distant. So we want the model to see this information about the spatial information that we see with our eyes.
For example, when we see this sentence – “what is positional encoding?”, we know that the word *what* is more far from the word *encoding* compared to other words, because we have this positional information given by our eyes. But the model cannot see this, so we need to give some information to the model about how the words are spatially distributed inside the sentence.
We want the positional encoding to represent a pattern that the model can learn, and we will see how.
Imagine we have sentence: “*your cat is a lovely cat”*. What we do is we first convert it into embeddings using the previous layer, so the input embeddings. These are embeddings of size 512.
Then we create some special vectors called the positional encoding vectors that we add to these embeddings. This vector we see here in red is a vector of size 512, which is not learned. It is computed once and not learned along with the training process - it is fixed.
This vector represents the position of the word inside of the sentence, and this should give us an output that is again a vector of size 512. This is because we are summing (the first dimension with the first dimension, the second dimension with the second dimension, and so on) . So we get a new vector of the same size as the input vectors.

Now, how are these positional embeddings calculated?

we have a smaller sentence, say “*your cat is”*, and you may have seen the following expressions from the paper.
What we do is create a vector of size d_{model}, so 512, and for each position in this vector we calculate the value using these two expressions and these arguments.
The first argument – “pos” indicates the position of the word inside the sentence. So the word “*your”* occupies position zero. for the even dimensions—0, 2, 4, 510, etc.—we use the first expression, the sine function. For the other positions of this vector, we use the second expression, the cosine function.
We do this for all the words inside the sentence. So this particular embedding is calculated as $PE(1,0)$because it’s the first word embedding, position zero. This value represents the argument *pos*, and the 0 represents the argument $2i$.
$PE(1,1)$means the first word, dimension one, so we will use the cosine, giving position one, and $2i + 1$will be equal to 1. We do this for the third word, and so on.
If we have another sentence, we will not have different positional encodings. We will have the same vectors even for different sentences, because the positional encodings are computed once and reused for every sentence that our model will see during inference or training.
So we only compute the positional encodings once when we create the model, we save them, and then we reuse them. We don’t need to compute them every time we feed a sentence to the model.

Why did the authors choose the cosine and sine functions to represent positional encodings?

Look at the plot of these two functions. You can see that the plot is shown by position, which is the position of the word inside the sentence, and the depth, which is the dimension along the vector—the 2i that you saw in the expressions.
If we plot them, we can see a clear pattern as humans, and we hope that the model can also see and learn this pattern.

Self-Attention

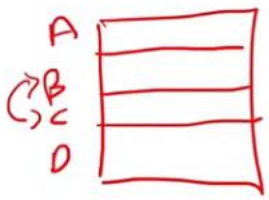
Self-attention is a mechanism that existed before the Transformer was introduced. The authors of the Transformer just changed it into multi-head attention.

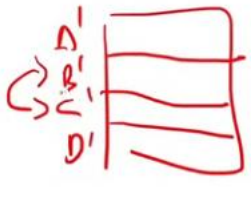
How self-attention works?
Self-attention allows the model to relate words to each other. We had the input embeddings that capture the meaning of the word, then we have the positional encoding that gives information about the position of the word inside the sentence. self-attention will relate words to each other.

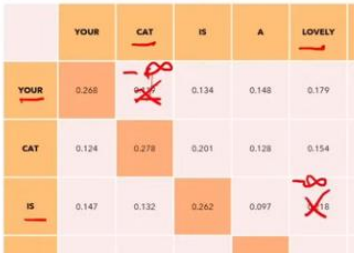
Now imagine we have an input sequence of six words with a d_{model} of size 512, which can be represented as a matrix that we will call Q, K, and V. So our Q, K, and V are the same matrix, representing the input: six words with dimension 512. Each word is represented by a vector of size 512.
Why self-attention? Because each word in the sentence is related to other words in the same sentence.
Q matrix - input sentence. we have six rows, and on the columns we have 512 columns.multiply Q by the transpose of K, which is again the same input sequence. We divide it by the square root of 512, and then we apply the softmax.
When we multiply a matrix that is 6 by 512 with another matrix that is 512 by 6, we obtain a new matrix that is 6 by 6. Each value in this matrix represents the dot product of the first row with the first column (represents the dot product of the embedding of the word “your” with itself), the first row with the second column (the dot product of the embedding of the word “*your”* with the embedding of the word “*cat”)*, and so on.
The SoftMax makes all these values sum up to one. Each row sums up to one.
These values in *softmax(*$\frac{QK\text{^}T}{\sqrt{d}}$ *)* represent a score indicating how strong the relationship is between one word and another.
we have multiplied Q by K, divided by the square root of d_k, and applied the softmax. Next, we multiply this matrix by V, and we obtain a new matrix that is 6 by 512.
If we multiply a matrix that is 6 by 6 with another that is 6 by 512, we get a new matrix that is 6 by 512. One important thing to notice is that the dimensions of this matrix are exactly the same as the dimensions of the initial matrix we started with.
This means we obtain a new matrix with six rows and 512 columns, where each row represents a word. Each word now has an embedding of dimension 512.
This embedding represents not only the meaning of the word given by the input embedding, and not only the position of the word added by the positional encoding, but also captures the relationship of this word with all the other words in the sentence.
So this embedding for each word captures its meaning, its position in the sentence, and its relationship with all the other words.

Self-attention has some desirable properties.
First, it’s permutation invariant. It means that if we have a matrix - let’s say we had a matrix of four words: *a*, *b*, *c*, and *d* - and suppose by applying the formula before this produces a particular matrix in which there is a new special embedding for word *a*, a new special embedding for word *b*, a new special embedding for word *c*, and for *d*. So let’s call them *a′*, *b′*, *c′*, *d′*.
If we change the position of two rows, the values will not change; the position of the output will just change accordingly. So the values of *b′* will not change, it will only change position, and *c′* will also change position, but the values in each vector will not change. This is a desirable property.

Self-attention, as described so far, requires no parameters. any parameter that are learned by the model are not introduced. just took the initial sentence—in this case six words—multiplied it by itself, divided it by a fixed quantity, which is the square root of 512, and then applied the softmax, which does not introduce any parameters. So for now, the self-attention layer does not require any parameters except for the embeddings of the words.
This will change later when we introduce multi-head attention.
Also, we expect that because each value in the self-attention - softmax matrix is a dot product of the word embedding with itself and with other words, the values along the diagonal will be the maximum. This is because they correspond to the dot product of each word with itself.
Before we apply the softmax, if we replace some values - for example, suppose we don't want the words '*your* and *cat*' to interact with each other, or we don't want the words '*is* and *lovely*' to interact with each other - we can replace those values with minus infinity.
When we then apply the softmax, minus infinity will be mapped to zero. As you remember, softmax uses e^x, and when x goes to minus infinity, e^xbecomes very close to zero.
This is a desirable property that we will use in the decoder of the Transformer.







Multi-head Attention

we are on the encoder side of the Transformer, and we have our input sentence, which is 6 by 512. This means six words by 512, which is the size of the embedding of each word.
In this case, I call it sequence(no of words) by d_{model}. Sequence is the sequence length, and $d_{model}$is the size of the embedding vector, which is 512.
we take this input and make four copies of it. One will be sent along [Add & Norm] connection, and three will be sent to the multi-head attention with three respective names. So it's the same input that becomes three matrices that are equal to the input: one is called query, one is called key, and one is called value. We call Q, K, and V. They have the same dimensions.

What does multi-head attention do? First, it multiplies these three matrices by three parameter matrices called W_Q, W_K, and W_V. These matrices have dimension $d_{model} \times d_{model}$. So if we multiply a matrix that is sequence by $d_{model}$with another one that is $d_{model} \times d_{model}$, we get a new matrix as output that is sequence by d_{model}, which is basically the same dimension as the starting matrix. We will call them Q', K', and V'.
Our next step is to split these matrices into smaller matrices. we can split this matrix Q': by the sequence dimension or by the d_{model} dimension. In multi-head attention, we always split by the d_{model}dimension.
So, every head will see the full sentence but a smaller part of the embedding of each word. If we have an embedding of, let's say, 512, it will become smaller embeddings of 512 divided by four,

and we call this quantity d_k. So $d_k = d_{model}/H$, where $H$is the number of heads. In our case, we have $H = 4$.
We can calculate the attention between these smaller matrices—Q_1, K_1, and V_1 - result Head1.
This will result in small matrices called Head 1, Head 2, Head 3, and Head 4.
The dimension of Head 1 up to Head 4 is sequence by d_v. d_v is equal to d_k. It's just called $d_v$because the last multiplication is done by V .
Our next step is to combine these small heads by concatenating them along the $d_v$dimension.
So we concatenate all these heads together, and we get a new matrix that is sequence by $H \times d_v$. Since $H \times d_v = d_{model}$, we get back the initial shape, which is sequence by d_{model}.

The next step is to multiply the result of this concatenation by W_O. $W_O$is a matrix of size $d_{model} \times d_{model}$, and the result of this multiplication is a new matrix that represents the output of the multi-head attention, which is again sequence by d_{model}.
So the multi-head attention, instead of calculating the attention directly between Q', K', and V', splits them along the d_{model}dimension into smaller matrices and calculates the attention between these smaller matrices.
Each head is watching the full sentence but a different aspect of the embedding of each word.
Why do we want this? Because we want each head to focus on different aspects of the same word. For example, in Chinese - but also in other languages - one word may be a noun in some cases, a verb in other cases, or an adverb in other cases, depending on the context.
So one head may learn to relate that word as a noun, another head may learn to relate that word as a verb, and another head may learn to relate that word as an adjective or adverb. This is why we want multi-head attention.
Attention can be visualized. When we calculate the attention between the Q and K matrices—when we apply the softmax to $QK^T/\sqrt{d_k}$ - we get a new matrix that is sequence by sequence.
This matrix represents a score that indicates the intensity of the relationship between two words.
We can visualize this, and it produces a visualization similar to the one taken from the paper, where we see how all the heads work.
For example, if we focus on the word "*making"*, we can see that "*making"* is related to the word "*difficult"* by different heads - the blue head, the red head, and the green head.
However, the violet (or pink) head does not relate these two words together. Instead, that head relates "*making"* to other words, for example to the word "*2009"*. This happens because that head is focusing on a different part of the embedding that the other heads do not see, which makes this interaction possible.

Why query, keys and values ?

So imagine we have a Python-like dictionary or a database in which we have keys and values.
The keys are the categories of movies, and the values are the movies belonging to that category. In my case, I just put one value.
So we have a *romantic* category, which includes *Titanic*. We have *action* movies that include *The Dark Knight*, etc.
Imagine we also have a user that makes a query, and the query is *love*. Because we are in the Transformer world, all these words are represented by embeddings of size 512.
So what our Transformer will do is convert this word *love* into an embedding of size 512. All these queries and values are already embeddings of size 512, and it will calculate the dot product between the query and all the keys, just like the formula.

The formula is the SoftMax of the query multiplied by the transpose of the keys, divided by the square root of the model size. So, we are doing the dot product of all the queries with all the keys - in this case, the word *love* with all the keys one by one.
This will result in a score that will amplify some values or not amplify other values. In this case, our embedding may be such that the word *love* and *romantic* are related to each other. The words *love* and *comedy* are also related to each other, but not as strongly as *love* and *romantic*, so the relationship is weaker. On the other hand, the word *horror* and *love* may not be related at all, so their SoftMax score would be very close to zero.

Layer Normalization

Layer normalization is a layer . Imagine we have a batch of *n* items. In this case, *n* is equal to three: item one, item two, item three.
Each of these items will have some features. It could be an embedding, for example a vector of size 512, but it could also be a very big matrix of thousands of features—it doesn't matter.
What we do is we calculate the mean and the variance of each of these items independently from each other, and we replace each value with another value that is given by expression. So basically, we are normalizing so that the new values are all in the range 0 to 1.
Actually, we also multiply this new value with a parameter called gamma, and then we add another parameter called beta. These gamma and beta are learnable parameters, and the model should learn how to multiply and add these parameters to amplify the values that it wants to be amplified and not amplify the values that it doesn't want to be amplified.
So we don't just normalize; we actually introduce some parameters.
see the difference between batch normalization and layer normalization. in layer normalization, if *n* is the batch dimension, we are calculating all the values belonging to one item in the batch. While in batch normalization, we are calculating the same feature for all the items in the batch. So in batch normalization, we are mixing values from different items of the batch, while in layer normalization we are treating each item in the batch independently, which will have its own mean and its own variance.

Batch Normalization (BN) and Layer Normalization (LN) both stabilize training by normalizing inputs, but differ in dimension: **BN normalizes across the mini-batch (per feature), while LN normalizes across all features for each input independently**. BN excels in CNNs with large batches; LN works best for RNNs/Transformers with small or variable batch sizes. Pinecone +4

Decoder
In the encoder, we saw the input embeddings. In Decoder, they are called output embeddings, but the underlying working is the same. Here also we have the positional encoding, and they are also the same as in the encoder.
The next layer is the masked multi-head attention. We also have another multi-head attention here. We should notice that the encoder produces an output that is sent to the decoder in the form of keys and values.

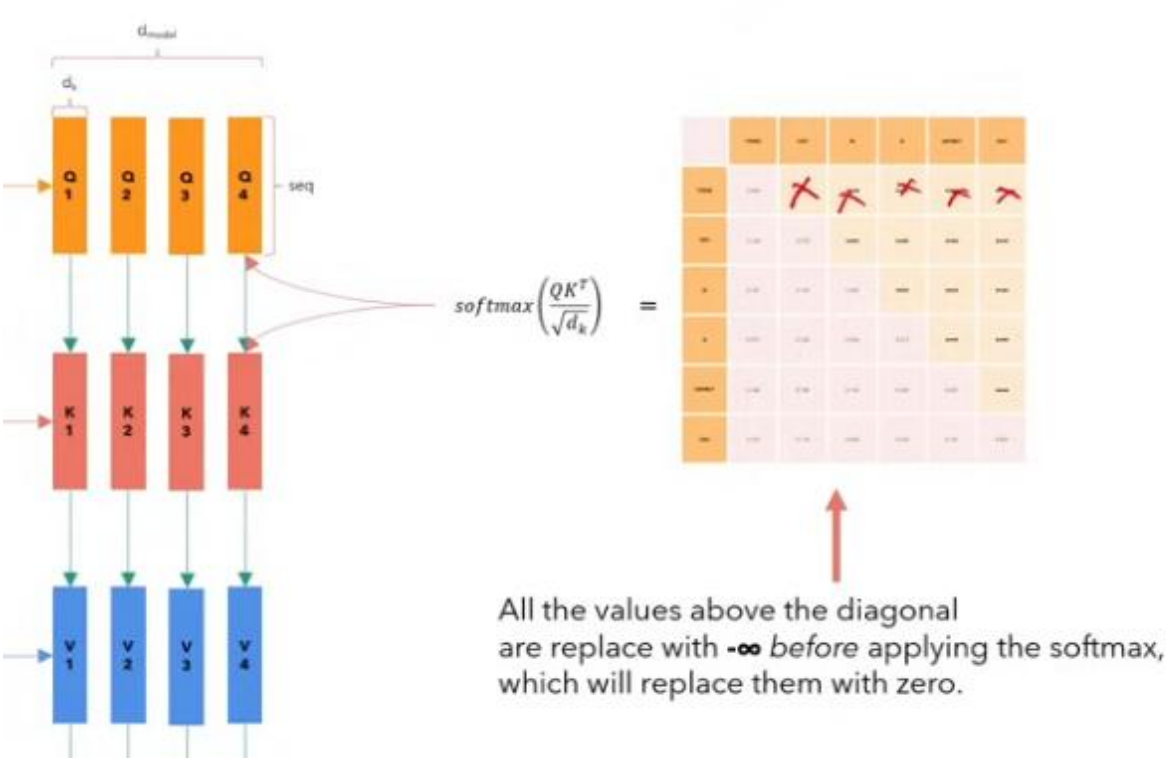
The query, on the other hand, is coming from the decoder. So, in this multi-head attention, it's not self anymore - it's cross-attention, because we are taking two different sequences. One comes from the encoder side, whose output is used as keys and values, while the output of the masked multi-head attention is used as the query in this multi-head attention.
The masked multi-head attention is the self-attention(multi) of the input sentence of the decoder. So we take the input sentence of the decoder, transform it into embeddings, add the positional encoding, and give it to this multi-head attention where the query, key, and value are the same input sequence.
Then we apply the add and norm layer, and we send this output as the queries to the multi-head attention, while the keys and values are coming from the encoder. Then we apply add and norm again.
the feed-forward network is just a fully connected layer. We then send the output of the feed-forward layer to another add and norm, and finally to the linear layer.

Masked multi-head attention

The masked multi-head attention makes the model causal. It means that the output at a certain position can only depend on the words in the previous positions, so the model must not be able to see future words.
How can we achieve that? As you saw, the output of the softmax in the attention calculation formula is a matrix of size sequence by sequence. If we want to hide the interaction of some words with other words, we delete those values and replace them with minus infinity before we apply the softmax, so that the softmax will replace these values with zero.

We do this for all the interactions that we don't want. For example, we don't want the word '*your*' to look at future words, so we don't want '*your*' to attend to '*cat*, *is*, *a*, *lovely*, or *cat*'. We also don't want the word '*cat*' to look at future words, but only at words that come before it or the word itself. The same applies to the other words.
So we are replacing all the values that are above the diagonal of this matrix. This diagonal is the principal diagonal of the matrix, and we want all the values above this diagonal to be replaced with minus infinity, so that the softmax will convert them to zero.

Which stage of the multi-head attention is this mechanism introduced? When we calculate the attention between these smaller matrices - so Q_1, K_1, and V_1 - before we apply the softmax, we replace some of these values - with minus infinity.
Then we apply the SoftMax, and the SoftMax will take care of transforming these values into zeros. So basically, we don't want these words to interact with each other.
If we don't want this interaction, the model will learn not to make them interact because the model will not get any information from this interaction. So it's like these words cannot interact.



Training

Our English sentence, is sent to the encoder. So our English sentence here, which we prepared, we append special tokens. One is called start of sentence and one is called end of sentence. These two tokens are taken from the vocabulary, so they are special tokens in our vocabulary that tell the model what is the start position of a sentence and what is the end of a sentence. we take our sentence, we prepend a special token, and we append a special token.

Then, we take our inputs, we transform them into input embeddings, we add the positional encoding, and then we send it to the encoder. So this is our encoder input: sequence by d_{model}. We send it to the encoder, and it will produce an output which is encoded as sequence by d_{model}, and it's called the encoder output.
the output of the encoder is another matrix that has the same dimension as the input matrix. We can see it as a sequence of embeddings, and this embedding is special because it captures not only the meaning of the word, which was given by the input embedding, and not only the position, which was given by the positional encoding, but also the interaction of every word with every other word in the same sentence. Because this is the encoder, we are talking about self-attention, so it's the interaction of each word in the sentence with all the other words in the same sentence.
Now we want to convert this sentence into Italian, so we prepare the input of the decoder, which starts with a start-of-sentence token. As you can see from the picture of the Transformer, the outputs here are shifted right. it means we prepared a special token called SOS, start of sentence.
You should also notice that when we code the Transformer, these sequences are made of a fixed length. We make the sequence of fixed length so that if we have a sentence that is '*te amo*

molto' or a very long sequence, when we feed them to the Transformer they all become the same length.
How do we do this? We add padding words to reach the desired length. So if our model can support, let's say, a sequence length of 1000, and in this case we have four tokens, we will add 996 padding tokens to make this sentence long enough to reach the sequence length.
We prepare this input for the decoder, we transform it into embeddings, we add the positional encoding, then we send it first to the masked multi-head attention along with the causal mask. Then we take the output of the encoder and send it to the decoder as keys and values, while the queries are coming from the masked attention. So the queries come from Add&Norm after Masked -attention, and the keys and the values are the output of the encoder.
The output of this whole block will be a matrix that is sequence by d_{model}, just like for the encoder. However, this is still an embedding because it has dimension d_{model}, so it is a vector of size 512.
How can we relate this embedding back into our dictionary? How can we understand what this word is in our vocabulary? That's why we need a linear layer that maps sequence by d_{model}into another sequence by vocabulary size. This tells us, for every embedding, what is the position of that word in our vocabulary, so we can understand what the actual output token by the model is. After that, we apply the SoftMax, and then we have our label - what we expect the model to output given this English sentence. We expect the model to output '*te amo molto'* and end of sentence. This is called the label or the target.
When we have the output of the model and the corresponding label, we calculate the loss. In this case, it is the cross-entropy loss, and then we backpropagate the loss to all the weights.
Now let's understand why we have these special tokens called SOS and EOS. Basically, you can see that here the sequence length is four - or actually 1000 because we have padding - but let's assume no padding. It's four tokens: start of sentence, *te*, *amo*, *molto*. What we want as output is *te*, *amo*, *molto*, end of sentence.
So when the model sees the start-of-sentence token, it outputs the first token *te*. When it sees *te*, it outputs *amo*. When it sees *amo*, it outputs *molto*. When it sees *molto*, it outputs end of sentence, which indicates that the translation is done. We will see this mechanism during inference.
All this happens in one time step. with recurrent neural networks we have *n* time steps to map an input sequence into an output sequence. That problem is solved with the Transformer. Because you can see here that we didn't do any for-loop. We just give an input sequence to the encoder and an input sequence to the decoder, produce the outputs, calculate the cross-entropy loss with the labels. Everything happens in one pass.
This is the power of the Transformer: it makes it very easy and very fast to train very long sequences with very strong performance, which you can see in models like GPT, BERT, and others.

Inference

We start with our English sentence "I love you very much", and we want to map it into an Italian sentence.
We use the usual Transformer model. First, we prepare the input for the encoder: "start-of-sentence, *I love you very much*, end-of-sentence". We convert this sequence into input embeddings and then add positional encoding. This prepared input is sent to the encoder.
The encoder produces an output sequence. This output consists of contextual embeddings that capture the meaning of the words, their positions, and their interactions with all other words in the sentence.

For the decoder, we initially provide only the start-of-sentence token. We add padding tokens as needed to match the sequence length. This single token is converted into embeddings, positional encoding is added, and it is sent to the decoder as input. The decoder uses this input as the query, along with the keys and values coming from the encoder output.
The decoder then produces an output sequence of dimension *d_model*. This output is passed through a linear layer that projects it back to the vocabulary space. The result of this projection is called logits. We then apply the softmax function to these logits to obtain probabilities over the vocabulary. The word with the highest probability is selected as the output token.
This process produces the first output token, for example “Ti”, if the model has been trained correctly. This happens at time step 1.
During training, the Transformer processes the entire input and output sequences in one pass. During inference, however, the process must be done token by token.
At time step 2, we do not recompute the encoder output because the English sentence has not changed. The encoder therefore produces the same output as before. What changes is the decoder input: we append the previously generated token (“Ti”) to the decoder input and feed it again into the decoder, along with the encoder output.
The decoder produces a new output, which is again projected onto the vocabulary and processed using SoftMax to obtain the next token, for example “amo”.
This process continues for time steps 3 and 4 in the same manner. At each step, the newly generated token is appended to the decoder input. The inference process stops when the model generates the end-of-sentence token, which signals that translation is complete.
This explains why inference requires multiple time steps, whereas training does not.

There are different strategies for inference. The one described above is called greedy decoding, where at every step we select the word with the maximum softmax probability. While this strategy often works reasonably well, there are better alternatives.
One such alternative is beam search. In beam search, instead of selecting only the single best token at each step, we keep the top B most probable tokens. For each of these choices, we explore possible next tokens and retain only the B most probable sequences, discarding the rest. This method usually produces better results than greedy decoding.

Public

Coding a Transformer from scratch on PyTorch, with full explanation, training and inference.

Introduction

Will be building a translation model which means that we our model will be able to translate from one language to another. I choose a data set that is called Opus books and it's a synthesis taken from famous books

Input Embeddings

The first part that we will be building is the input embedding. the input embeddings take the input and convert it into an embedding. The input embeddings allow us to convert the original sentence into a vector of 512 dimensions.
For example, in this sentence, *"your cat is a lovely cat"*, first we convert the sentence into a list of input IDs. These are numbers that correspond to the position of each word inside the vocabulary, and then each of these numbers correspond to an embedding, which is a vector of size 512.
PyTorch already provides a layer that does InputID to vector. Given a number(input ID), it will provide you with the same vector every time, and this is exactly what an embedding does. It's just a mapping between numbers and a vector of size 512. The 512 here, is the d_model, and this is done by the embedding layer.
Embedding layer,is just a dictionary-kind of layer that maps numbers to the same vector every time. This vector is learned by the model, so we just multiply this by the square root of *d_model*.

embedding layer we multiply the weights of the embedding by square root of D model

Positional Encodings

our original sentence gets mapped to a list of vectors by the embedding layer .
we want to do is convey to the model information about the position of each word inside the sentence. This is done by adding another vector of the same size as the embedding, so of size 512. This vector includes some special values given by a formula, which tells the model that this particular word occupies this position in the sentence.
So, we will create these vectors, called positional embeddings, and we will add them to the embeddings.

dropout is to make the model less overfit
Positional encoding is a matrix of shape *sequence length* × *d_model*. we need vectors of size *d_model* - 512. We need *sequence length* number of these vectors because the maximum length of the sentence is the sequence length.

We create a vector of size 512 for each possible position up to the maximum sequence length. Sine functions are used for the even positions, while cosine functions are used for the odd positions. There is also a batch dimension representing the number of sentences. We add this positional encoding to every word in the sentence. We do not want to learn this positional encoding because it is fixed and will always remain the same, so it is not updated during training. Therefore, we set requires_grad to False, which ensures that this tensor is not learned during the training process.

Layer normalization

when you have a batch of *n* items — for example, three items — each item has a set of features. You can think of these items as sentences, where each sentence is made up of many words represented numerically. For layer normalization, we treat each item in the batch independently. For every item, we compute its own mean and variance, without considering the other items in the batch. Using this item-specific mean and variance, we normalize the values of that item.
In layer normalization, two additional learnable parameters: one multiplicative parameter and one additive parameter. These are commonly called gamma and beta, though some people refer to them as alpha and beta or alpha and bias. The multiplicative parameter (gamma or alpha) is multiplied with the normalized values, and the additive parameter (beta or bias) is added afterward. The additive term is always added, and the multiplicative term controls how much each value is scaled.
These parameters allow the model to adjust the scale and shift of the normalized values. This gives the model the flexibility to amplify certain values when needed. During training, the model learns how to scale the values using the multiplicative parameter in a way that highlights the features it considers important.
epsilon, which is a very small number added to the denominator during normalization. This is required for numerical stability. If the variance (sigma) becomes zero or very close to zero, the denominator would become extremely small, causing the normalized values to grow very large. Since CPUs and GPUs can only represent numbers within a limited range, this can lead to numerical issues. Adding epsilon prevents division by zero and avoids extremely large or unstable values.

3.3 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2}$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_{\text{model}} = 512$, and the inner-layer has dimensionality $d_{ff} = 2048$.

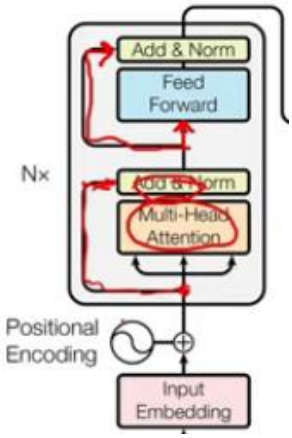
Feed-Forward Network
The feed-forward layer is essentially a fully connected neural network that is applied independently to each position in the sequence.

Multi-Head Attention in Transformers
In the encoder, we use multi-head self-attention. The same encoder input is used three times: once as the query, once as the key, and once as the value. applying the same input to three different roles.
The multi-head attention mechanism works as follows:
We start with an input sequence of shape (sequence length × d_model). From this input, we generate three matrices—Q (queries), K (keys), and V (values). In the encoder, these matrices initially come from the same input. We then multiply each of them by learning weight matrices W_Q, W_K, and W_V, which results in new matrices that are still of dimension (sequence length × d_model).

Next, we split these matrices into H smaller matrices, where H is the number of attention heads. This split is performed along the embedding dimension, not the sequence dimension. As a result, each attention head has access to the full sequence, but only to a portion of the embedding features of each word.
We then apply the attention mechanism independently to each of these smaller matrices using the scaled dot-product attention formula. Each head produces its own output matrix. After that, we concatenate the outputs of all heads back together. Finally, this concatenated matrix is multiplied by another learned matrix W_O to produce the final multi-head attention output.
The output of the multi-head attention has the same dimensionality as the input—that is, (sequence length × d_model).
In Transformers, we do not operate on a single sentence at a time. Instead, we process multiple sentences simultaneously, which introduces an additional batch dimension. Therefore, the full input typically has the shape (batch size × sequence length × d_model).
When splitting the embedding vector into H heads, it is required that d_model is divisible by H. Otherwise, we would not be able to divide the embedding space evenly across all heads.

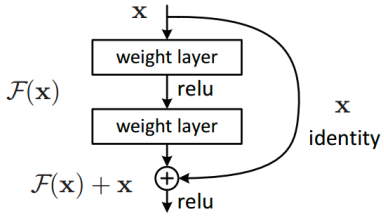
When we multiply the query (Q) matrix by the key (K) matrix, we obtain a matrix that represents how each word interacts with every other word. This results in a sequence-by-sequence matrix, where each entry corresponds to the attention score between two tokens in the sequence.
If we do not want certain words to interact with other words, we modify their attention scores before applying the softmax function. Specifically, we replace those values with very large negative numbers.
When softmax is applied, these large negative values effectively become zero. This happens because the softmax function uses e^x in its numerator, and when x approaches negative infinity, e^x becomes extremely small—very close to zero.
As a result, the attention for those word pairs is effectively hidden. This mechanism is exactly the purpose of the attention mask: it prevents certain tokens from attending to others by forcing their attention weights to zero.

The query is multiplied by the transpose of the key and divided by the square root of d_k. The result of this operation gives us the **attention scores**.



Residual Connection (Skip Connection)

Residual connections (or skip connections) in transformers are shortcuts that add the input of a layer directly to its output, calculated as Output = Layer(Input) + Input .



Residual Connection (Skip Connection)
A **residual connection** is the connection between a layer's input and its output after passing through another layer and normalization. For example, we take the output of a layer—such as

the multi-head attention—and combine it with the original input through an Add & Norm operation.
To implement this, we create a layer that manages the skip connection. The idea is simple:

- We take the input x
- We pass $x$through the previous layer (called the sublayer, such as multi-head attention)
- We apply dropout to the sublayer output
- We add the original input $x$back to this result
- Finally, we apply layer normalization

This is why it is often called a skip connection: the input skips over one layer and is added directly to that layer's output.

To build this residual connection layer, we define a constructor where we initialize:

- A dropout layer
- A layer normalization module

In the forward method, we take the input $x$and the sublayer (the previous layer's operation). We compute the output by combining $x$with the sublayer's output after applying dropout.
This operation represents the Add & Norm step.

In the original Transformer paper, the order is:

1. Apply the sublayer
2. Add the residual connection
3. Apply normalization

However, in many modern implementations, the order is slightly different:

1. Apply normalization
2. Apply the sublayer
3. Add the residual connection

Since many implementations successfully use this variant, we follow the same approach in code.

Source Mask
The source mask in encoder : We need a mask for the input of the encoder because we want to hide the interaction of the padding word with other words. We do not want the padding word to interact with other words, so we apply the mask.
Encoder Block: self-attention + normalization + residual connection

Decoder
In the decoder, the output embeddings class is the same as the input embeddings class. we use the same values of positional encodings that we use for the encoder also for the decoder.

Decoder block is made of masked multi-head attention, add & norm, then another multi-head attention with another skip connection, and finally the feed-forward network with its skip connection.

we have the same input used three times in the masked multi-head attention (self-attention). The same input plays the role of the query, the key, and the value, which means that each word in the sentence is matched with every other word in the same sentence.
In the next attention block, we calculate attention using the query coming from the decoder, while the key and the values come from the encoder. this is called **cross-attention**, because we are crossing two different objects together and matching them to calculate the relationship between them.

We have the **source mask**, which is the mask applied to the encoder, and the **target mask**, which is the mask applied to the decoder. They are called source mask and target mask because, in this case, we are dealing with a translation task. We have a source language (for example, English) and a target language (for example, Italian).
the source mask is the one coming from the encoder (the source language), and the target mask is the one coming from the decoder (the target language).

```
x = self.residual_connections[0](x, lambda x: self.self_attention_block(x, x, x, tgt_mask))
x = self.residual_connections[1](x, lambda x: self.cross_attention_block(x, encoder_output, encoder_output, src_mask))
```

Linear Layer

the output to be of dimension (seq_len, d_model) if we do not consider the batch dimension is coming from the Decoder. However, we want to map these word representations back into the vocabulary space.
For this reason, we need a linear layer that converts each embedding vector into a vocabulary-sized vector. This layer is usually called the projection layer, because it projects the embedding space into the vocabulary space.
In other words, it maps the tensor from (seq_len, d_model) → (seq_len, vocab_size).

Then Softmax is applied to the output tensor of Linear Layer

Transformer

Transformer block consists of an encoder and a decoder. We have the encoder , decoder , a source embedding and a target embedding.
We need both source and target embeddings because we are dealing with multiple languages. We have one input embedding for the source language and one input embedding for the target language.
The source and target positional encoding layers do the same job.
We create the projection layer to convert the model output into the vocabulary size. This vocabulary is the target vocabulary, because we want to translate from the source language to the target language. So we project the output into the target vocabulary.
Finally, we use Xavier uniform initialization. This is a method used to initialize the model parameters so that training is faster and more stable, instead of starting from completely random values.

Dataset

For the translation task, chosen a dataset called Opus Books, found on Hugging Face. We use the Hugging Face tokenizer library to transform this text into a vocabulary.
We are using the subset English to Italian. we build the code in such a way that you can choose the language and the code will act accordingly.
If we look at the data, each data item is a pair of sentences: one in English and one in Italian.

Tokenizer

The tokenizer is what comes before the input embeddings.
The goal of the tokenizer is to create tokens, meaning it splits the sentence into single words.
There are different types of tokenizers: BPE tokenizers, word-level tokenizers, sub-word tokenizers, and others.

we will use word-level tokenizer. The word-level tokenizer basically splits the sentence by spaces, so each space defines the boundary of a word.
Each word is then mapped to a number. This is the job of the tokenizer: to build the vocabulary of these numbers and to map each word into a corresponding number.

word-level trainer - splits words using whitespace and works on single words. It also includes four special tokens. One is unknown, which is used when a word cannot be found in the vocabulary and is replaced with the unknown token. Another is the padding token, which we use to train the Transformer. We also have the start-of-sentence (SOS) token and the end-of-sentence (EOS) token. There is also a minimum frequency requirement: for a word to appear in the vocabulary, it must have a frequency of at least two.

We need the start-of-sentence token, the end-of-sentence token, and the padding token. To convert special tokens like the start-of-sentence token into a number that can be used as an input ID, the tokenizer provides a special method to do that.
The tokenizer will first split the sentence into single words and then map each word to its corresponding number in the vocabulary.
We then pad the sentence to reach the fixed sequence length. This is very important because the model always works with a fixed sequence length, but not every sentence has enough words. We use the padding token to fill the sentence until it reaches the required sequence length.

In the encoder input, we include both the SOS and EOS tokens. In the decoder input, we include only the SOS token. For the labels, we add only the EOS token.
The sequence length we choose must be large enough to represent all the sentences in the dataset. If we choose a sequence length that is too small, we raise an exception. This means that the number of padding tokens should never become negative.
The decoder input starts with the SOS token, and the label ends with the EOS token, which is what we expect as output from the decoder.

For the encoder mask, we increase the size of the encoder input sentence by adding padding tokens, but we do not want these padding tokens to participate in self-attention. Therefore, we build a mask that prevents these padding tokens from being seen by the self-attention mechanism.
For the decoder, we need a special mask called a causal mask. This mask ensures that each word can only look at previous words and not future ones. We also ensure that padding tokens do not participate in self-attention. Only real words are allowed to interact, and each word can attend only to the words that come before it.

A causal mask means that we want each word in the decoder to only look at the words that come before it. in the decoder each token should not have access to future tokens.
The matrix we are masking represents the multiplication of the queries by the keys in the self-attention mechanism. What we want to do is hide all the values above the diagonal of this matrix.
For example, the word ‘*cat’* in the sentence “your cat is a lovely cat” can only attend to itself and the words that come before it. The word ‘*lovely’* can attend to all the words that appear before it, from ‘*your’* up to ‘*lovely’* itself, but it cannot attend to the word ‘*cat’* that comes after it.

sequence length —the maximum length of each sentence in the source and the target
seq_length = length of longest sentence in both source and target + sos + eos

TensorBoard allows us to visualize the loss, the graphics, the charts
We use Adam optimizer.
loss function we will be using is the cross entropy loss
we need to tell loss func what the ignore index is, so we want it to ignore the padding token - we don't want the loss from the padding token to contribute to the loss

Label smoothing - label smoothing basically allows our model to be less confident about its decision
so imagine our model is telling us to choose word number three with a very high probability
so what we will do with label smoothing is take a small percentage of that probability and distribute it to the other tokens
so that our model becomes less sure of its choices, kind of less overfit, and this actually improves the accuracy of the model . From every highest probability token take some percent of score and give it to the others

use batch iterator

Cross-Entropy – loss function

It is very good idea when we want to be able to resume the training to also save not only the state of the model but also the state of the optimizer because the optimizer also keeps track of some statistics, one for each weight, to understand how to move each weight independently.
If you want your training to be resumable you need to save it. Otherwise, the optimizer will always start from zero and will have to figure out from zero how to move each weight, even if you start from a previous epoch.

We also would like to visualize the output of the model while we are training, and this is called validation. so we want to check how our model is evolving while it is getting trained.
what we want to build is a validation loop which will allow us to evaluate the model, which also means that we want to perform inference from this model and check some sample sentences and see how they get translated.

So, with torch.no_grad, we are disabling the gradient calculation for every tensor. During inference from the model, we don't want to train it. For the validation dataset, we only have a batch size of one - infer only with one sample

When we want to infer the model, we need to calculate the encoder output only once and reuse it for every token that the decoder will output. Greedy decoding on model - runs the encoder only once and reuses the result for every decoding step.
We pre-compute the encoder output and reuse it for every token generated by the decoder. We start by giving the decoder the start-of-sentence token so that it can output the first token of the translated sentence. Then, at every iteration - we append the previously generated token to the decoder input. This allows the decoder to output the next token. We then take that new token, append it again to the input, and continue generating successive tokens. We keep asking the decoder to output the next token until we either reach the end-of-sentence token or hit the maximum length we defined.
We use a causal mask to ensure that the decoder cannot look at future tokens. We don't need any other mask here because we don't have padding tokens in this case. We reuse the encoder output in every iteration of the loop, along with the source mask (the encoder mask). Then we pass the decoder input together with its mask (the decoder mask) to obtain the next token. We compute the probability of the next token using the projection layer, but we only care about the

last token's projection - the next token after the sequence we have already given to the decoder. We then select the token with the highest probability—this is greedy search. We append this token to the sequence, since it will serve as input for the next iteration. In practice, we concatenate the decoder input with the newly generated token. Finally, if the generated token is equal to the end-of-sentence token, we stop the loop. And that's our greedy decoding process.

What we give as input to the model, what the model outputs (the predictions), and what we expected as the output—we print all of them. To get the text form of the model's output, we need to use the tokenizer again to convert the tokens back into text.

Attention visualization

We have attention from different parts of the model. For example, there are three main places where attention appears:

- First, in the **encoder**
- Second, in the **decoder**, at the beginning (the decoder's self-attention)
- Third, the **cross-attention** between the encoder and the decoder

So, we can visualize three types of attention.
Also, because we use multiple attention heads, each head focuses on a different aspect of each word. This is because we distribute the word embedding across the heads equally, so each head sees a different part of the embedding. As a result, each head can learn different kinds of relationships between words.